(E) We flip the bits that were still set to `0`: in this example, the ones at indices `2` and `9`. Now all the bits to which "`duffy@acme.com`" hashes to are set to `1`, and so any future lookup will return `true`.

In listing 4.11 you can see the reason why, when we wrote method `contains`, we added the possibility to pass it an array with the pre-computed bit indices for the current key: inside `insert`, we perform a read closely followed by a write. The operation to compute all the indices for the bits of a key can be expensive, so to avoid repeating this operation twice at short distance, we need a way to pass its result to contains. At the same time, we don't want to add this option to our *API*, since this is internal magic that clients need not to know about: if your programming language supports polymorphism and private methods, restricting the optional parameter to a private version of `contains` (internally called by the public one) would be wise.

Another possible approach to save this duplicated effort could be having `writeBit` checking if the bit overridden was already set to `1`, and return a Boolean to state if the bit's value has changed. Then `insert` could check if at least one bit was flipped. This alternative implementation will be provided in our repo: it's up to you to decide which one you consider the cleanest.

Either way, striving to count unique keys added to the filter is going to be costly. Will the overhead be justified? That depends: if you don't expect many duplicate keys added to your filter, then probably it's not worth it. But just know that